

What Transfers from Text-to-SQL to Text-to-Cypher?

A Controlled Study of Reasoning, Selection, and Training Recipe

Cyrus Parsons

Abstract

Text-to-SQL has produced a large toolbox of techniques—chain-of-thought (CoT) distillation, self-consistency, execution-based selection, reinforcement learning—but it is unclear which of them transfer to the structurally different task of Text-to-Cypher. We run a controlled, matched-pipeline study to find out, unified by a single *constrained-output-space hypothesis*: a Cypher graph pattern is a single *connected* object with few semantically-equivalent surface forms, whereas SQL is *compositional* with many. This predicts that (P1) string-diversity methods fail, (P2) execution-grounded methods still transfer, and (P3) CoT/decomposition hurts. Our central result is negative and, to our knowledge, the first controlled demonstration of it: CoT distillation—the highest-profile SQL transfer—does *not* improve Text-to-Cypher and modestly hurts it. The effect holds across two model families (Gemma-2-9B, Llama-3.1-8B), naive and execution-verified traces, and in-distribution, compositional, and length-generalization regimes, with paired-bootstrap confidence intervals excluding zero throughout. Matched controls extend the negative to SPARQL and—under the literature’s own execution-accuracy metric, on Spider—to SQL itself (−0.099): against a strong completion-only SFT recipe, reasoning distillation subtracts everywhere we measure, consistent with a *baseline-strength* account of published CoT gains, while output-space structure governs the failure modes and the remedies. Apparent gains under standard evaluation practice—comparing against a published fine-tuned baseline—are attributable to a stronger supervised fine-tuning recipe and a leaked benchmark split, not to reasoning. We give a causal mechanism: CoT’s decompositional bias fragments connected graph patterns, and a holistic-reasoning ablation recovers ~40% of the penalty by cutting fragmentation 5–6×. Consistent with the hypothesis, execution-grounded selection transfers (+1.1pp exact match) while string self-consistency does not. We close with a methodological lesson: a published artifact used as a control, rather than a matched in-pipeline baseline, yields a large and fully spurious “CoT helps” result.

1 Introduction

Relational databases have a mature natural-language interface problem, Text-to-SQL, backed by a decade of intensive work. Graph databases pose the younger sibling of that problem. Systems such as Neo4j store entities and their relationships as first-class graph structure and are queried in Cypher, whose central construct is a drawn path such as `(p:Person)-[:ACTED_IN]->(m)`. Graph workloads—fraud rings, recommendation networks, knowledge graphs—are exactly the settings where non-expert users most want to ask questions in natural language, and the Neo4j Text2-Cypher benchmark [Ozsoy et al., 2025] has made the task measurable at scale. The economically interesting regime is small open-weight models: an 8–9B model that translates reliably can be served cheaply and privately, where frontier-model APIs cannot. But small models need every trick available, which raises the question this paper is about: *where should those tricks come from?*

The obvious source is Text-to-SQL, which offers a well-stocked toolbox with headline numbers attached. Chain-of-thought (CoT) distillation—fine-tuning a small model on teacher-written rea-

soning traces rather than bare queries—is the highest-profile item: STaR-SQL [He et al., 2025] reports 86.6% Spider execution accuracy, an 18-point improvement over direct-answer fine-tuning. Around it sit self-consistency [Wang et al., 2022], execution-based candidate selection, decomposition pipelines [Tai et al., 2023, Pourreza and Rafiei, 2023], and execution-rewarded reinforcement learning [Shao et al., 2024, Yao et al., 2025a]. Prior Text-to-Cypher work has begun importing these techniques one at a time—RL [Tran et al., 2025], iterative refinement [Ozsoy, 2025a], schema filtering [Ozsoy, 2025c]—and reporting absolute numbers. What no prior work provides is a *matched-pipeline* test of the transfer assumption itself: a comparison in which a single technique is the only variable, against a baseline trained in the same pipeline. This paper asks, in that controlled setting, *which techniques actually transfer, and why*.

There is a structural reason to doubt that transfer is automatic. A SQL query *composes*: subqueries, CTEs, and join clauses are separable parts that can be drafted independently, and the same semantics admits many surface forms. A Cypher basic graph pattern is a single *connected* object—every node variable is shared between adjacent relationship atoms—that must be assembled as a whole, and for a given schema and question it typically has one canonical surface form. We develop this contrast into the **constrained-output-space hypothesis** (§3), which yields three testable predictions: string-diversity methods should fail (P1), execution-grounded methods should still transfer (P2), and decomposition-style reasoning should actively hurt (P3).

The study is framed by a methodological demonstration. Standard practice in this space is to measure a new technique against a *published* fine-tuned baseline, run through one’s own evaluation harness. For CoT distillation on the Neo4j benchmark, that comparison yields a large and stable apparent gain (+0.12 GLEU)—large enough to anchor a paper, and reinforced by every downstream analysis built on top of it. Training the matched direct-answer control in the same pipeline reverses the sign: the apparent gain decomposes into +0.14 from the training pipeline and −0.017 from CoT itself (Table 2). A second standard practice, verifying train–test hygiene by instance ID alone, likewise manufactures an apparent length-generalization state-of-the-art on ZOGRASCOPE [Cazzaro et al., 2025] that dissolves under content-level auditing (58.6% train–test overlap). Both practices are common, and both produce publishable illusions; we therefore report them as results rather than footnotes.

Our study is built so that neither confound can recur. Every comparison fixes the base model, the QLoRA configuration, the loss masking, the teacher, and the decoding, and varies *only* the training target: the query alone (*direct*) versus a reasoning trace followed by the query (*CoT*). We instantiate this protocol across two model families (Gemma-2-9B, Llama-3.1-8B) and three query formalisms (Cypher, SPARQL, SQL), score with both translation and execution metrics on leakage-verified splits, and attach paired-bootstrap confidence intervals to every central delta.

The verdict is uniform. CoT distillation degrades Text-to-Cypher on every metric, for both model families, with naive and with execution-verified traces, in-distribution and under compositional and length generalization; matched controls return the same sign for SPARQL and—under the literature’s own execution-accuracy metric, on Spider—for SQL itself. Twelve deltas, all negative, all with confidence intervals excluding zero. The Spider result carries particular weight: it closes the possibility that our SQL negative was a string-metric artifact, and it removes the expected sign flip that a purely structural account would predict. Our best-supported explanation is *baseline strength*: our completion-only training recipe produces direct baselines strong enough to erase reasoning’s headroom—our plain direct-SFT model reaches 76.7% Spider dev execution accuracy, above the 75.0% STaR-SQL reports for its CoT-fine-tuned counterpart—suggesting that published CoT-SFT gains partly measure the weakness of their direct-answer controls.

What the structural hypothesis does govern, we show, is *how* CoT fails and *which* remedies work. On execution-grounded diagnostics, 97% of CoT’s failures identify the correct relationships

but assemble them into the wrong connected structure; a holistic-reasoning ablation that removes decomposition recovers $\sim 40\%$ of the penalty by cutting pattern fragmentation $5\text{--}6\times$, establishing the mechanism causally, and the residual traces to error accumulation (filter values corrupted during reasoning, then faithfully copied into the query). On the remedy side, execution-result selection transfers exactly as P2 predicts—clustering sampled candidates by their result sets beats both greedy decoding and string voting—and the mundane recipe choice of completion-only loss masking transfers with force. A final methodological finding rounds out the study: neither the published baseline artifact (from its documented recipe) nor its published score (from its documented inference) proved reproducible, so cross-harness absolute comparisons are untrustworthy in this space, and within-pipeline deltas are the only safe currency.

Contributions.

1. **A controlled matched-baseline result** that CoT distillation does not help, and modestly hurts, Text-to-Cypher—across Gemma-2-9B and Llama-3.1-8B, naive and execution-verified traces, and in-distribution / compositional / length-generalization regimes—and that the negative extends under matched controls to SPARQL and to SQL itself (Spider, execution accuracy) (§5.1, §5.3).
2. **A causal mechanism:** decomposition fragments connected graph patterns; a holistic-reasoning ablation recovers $\sim 40\%$ of the penalty (§6).
3. **A positive transfer:** execution-grounded (MBR) selection beats string voting while string self-consistency fails, confirming the hypothesis (§5.4).
4. **A methodological lesson:** always train a matched in-pipeline baseline before attributing a delta to an intervention (§7).

Organization. §3 states the hypothesis and its predictions; §4 specifies the matched protocol, metrics, and leakage discipline. §5.1–5.2 present the matched negative result and its robustness checks, §5.3 the cross-formalism controls (SPARQL, SQL, Spider), and §5.4 the techniques that do transfer, including the training-recipe decomposition and its reproducibility findings. §6 establishes the mechanism causally, and §7 develops the baseline-strength account and the methodological lessons.

2 Related Work

2.1 The Text-to-SQL toolbox

The techniques whose transfer we test were each developed and validated on Text-to-SQL.

Reasoning and decomposition. Chain-of-thought prompting [Wei et al., 2022] generates intermediate reasoning before the answer. For Text-to-SQL, Tai et al. [2023] systematically compared CoT strategies and found QDecomp+InterCOL—question decomposition with intermediate schema-element identification—most effective (+5.2pp on Spider dev with Codex). *Distilling* such reasoning into small models is the highest-profile transfer candidate: STaR-SQL [He et al., 2025] reports 86.6% Spider execution accuracy from an iterative rationale-bootstrapping loop, and Rossiello et al. [2025] distill CoT rationales for Text-to-SQL directly. Two details of the SQL evidence matter for what follows. First, the clean same-data comparison inside STaR-SQL (CoT-SFT vs. direct-SFT) is +6.4pp; the celebrated +18pp includes a trained verifier doing best-of-16 selection. Second, several strong results bundle CoT with other machinery: Struct-SQL [Thaker and Bresler, 2025]

structures the reasoning around execution plans, ExCoT [Yao et al., 2025b] adds iterative DPO, and Liu et al. [2025] show DPO *requires* CoT to work for Text-to-SQL. Decomposed pipelines (DIN-SQL Pourreza and Rafiei 2023; DTS-SQL Pourreza and Rafiei 2024; CHASE-SQL Pourreza et al. 2025a) make the same bet at inference time. The theoretical basis for decomposition is QDMR [Wolfson et al., 2020, 2022].

Sampling, selection, and RL. Self-consistency [Wang et al., 2022] samples multiple reasoning paths and majority-votes the answer; execution-grounded variants cluster candidates by their *result sets* rather than their strings. Agentic refinement loops (MAC-SQL Wang et al. 2025) add error-driven correction. GRPO [Shao et al., 2024] is the dominant RL algorithm for query generation: Arctic-Text2SQL-R1 [Yao et al., 2025a] reaches #1 on BIRD with execution-only rewards, and Reasoning-SQL [Pourreza et al., 2025b] uses composite partial rewards.

2.2 Text-to-Cypher

Compared to Text-to-SQL [Mohammadjafari et al., 2024, Liu et al., 2024], Text-to-Cypher is younger. Early systems were rule- or template-based [Kobeissi et al., 2021, Nie et al., 2022]. The Neo4j Text2Cypher benchmark [Ozsoy et al., 2025]—44,387 question–schema–Cypher triplets over 966 schemas—established the standard evaluation: fine-tuned GPT-4o leads at 0.8017 GLEU, and the benchmark’s fine-tuned Gemma-2-9B baseline reports 0.5560. Follow-up work explores schema filtering [Ozsoy, 2025c], data pruning [Ozsoy, 2025b], iterative refinement [Ozsoy, 2025a], and multilingual adapters [Ozsoy, 2026]. Auto-Cypher [Tiwari et al., 2025] generates fully-executable synthetic training data (using CoT during *generation*, not at inference); Yang et al. [2026] combine synthetic data, preference learning, and schema retrieval; Hornsteiner et al. [2024] build a live-schema pipeline with error-correction loops; Feng et al. [2023] chain prompts for a Chinese KBQA task. Closest to our question, Tran et al. [2025] apply GRPO with structured-extraction rewards to Qwen2.5-3B (GLEU 0.7701 on Neo4j 2024)—the first RL for Text-to-Cypher—and STRuCT-LLM [Stoisser et al., 2025] trains jointly on SQL and Cypher with topology-aware rewards. Benchmarks beyond Neo4j’s include CypherBench [Feng et al., 2025], ZOGRASCOPE [Cazzaro et al., 2025]—a manually annotated Pole-graph benchmark with IID, compositional, and length-generalization splits—and Mind the Query [Chauhan et al., 2025].

2.3 Positioning

Prior Cypher work *imports* individual SQL techniques (RL, refinement loops, schema filtering) and reports absolute numbers, implicitly assuming the toolbox transfers. None runs matched-pipeline controls in which a single technique is the only variable, and the highest-profile transfer of all—CoT distillation, assumed positive on the strength of the SQL literature—has not been tested against a matched direct-answer baseline at all. That is the gap this paper fills; our matched controls overturn the assumed-positive for CoT while confirming transfer for execution-grounded selection.

3 The Constrained-Output-Space Hypothesis

The structural difference between the two query languages can be stated precisely. A SQL query *composes*: sub-queries, CTEs, and join clauses are separable units that can be drafted independently and then combined, and the same semantics admits many surface realizations—JOIN vs. correlated subquery, alias renaming, clause reordering, IN vs. EXISTS. A Cypher query is dominated by its basic graph pattern, e.g. (a)-[:R]->(b)-[:S]->(c), which is a single *connected* object: every

node variable is shared between adjacent relationship atoms, so no sub-part can be produced independently without violating a global consistency constraint. The pattern must be assembled as a whole, and—for a given schema and question—it typically has one canonical surface form (Neo4j’s own training data reinforces this canonicity).

Write $\mathcal{Y}(q)$ for the set of correct surface forms for question q . The hypothesis is twofold: $|\mathcal{Y}_{\text{Cypher}}(q)| \ll |\mathcal{Y}_{\text{SQL}}(q)|$, and correct Cypher outputs are *globally coupled* (connected patterns) where SQL outputs are *locally decomposable*. Three predictions about the SQL toolbox follow:

- **P1 (string diversity fails).** With essentially one canonical form, sampling produces little useful diversity, so string-vote self-consistency cannot help.
- **P2 (execution grounding transfers).** Methods that operate on *results* bypass the surface-form bottleneck and still help.
- **P3 (decomposition hurts).** Decomposing a connected pattern fragments it: whatever value decomposition has for SQL’s separable clauses, it is the wrong inductive bias for Cypher. (P3 predicts harm for Cypher; it is agnostic about the sign for SQL, which §5.3 measures directly.)

A corollary sharpens P3. When the benchmark supplies entity values explicitly (as ZOGRASCOPE does), query generation is largely a *copy-and-place* task: the values are given, and the work is placing them in the right structural slots of one connected pattern. Direct-answer generation copies each value once. Reasoning-first generation re-states values, paths, and structure in prose before the query, and every re-statement is an opportunity to corrupt, drop, or add—so the hypothesis predicts not just that decomposition mis-assembles structure but that longer reasoning chains *accumulate* errors on such tasks. Both effects appear in §6.

4 Matched Experimental Protocol

Every comparison in this paper fixes the base model, the QLoRA configuration (4-bit NF4, $r=64$, $\alpha=64$, all-linear targets, `paged_adamw_8bit`), the completion-only loss masking, and the CoT-distillation procedure (traces from GPT-oss-120B), and varies *only* the training target: the reference query alone (*direct*) versus a reasoning trace followed by the reference query (*CoT*). Both arms train on the identical set of instances (those with a valid trace), so no data differs between them. We instantiate this protocol on Cypher (Neo4j 2024; ZOGRASCOPE), SPARQL (LC-QuAD 2.0), and SQL (§5.3).

Trace generation. Reasoning traces are distilled from GPT-oss-120B (a 117B-parameter open-weight MoE reasoning model). For the primary Neo4j arm we generated traces for all 39,554 training examples (99.997% success; $\sim\$50$). Each trace follows a four-step QDecomp+InterCOL decomposition adapted from SQL [Tai et al., 2023] to graphs: (1) decompose the question into sub-questions; (2) link each to schema elements (node labels, relationship types, properties); (3) identify the traversal pattern; (4) construct the query clause by clause. The reference query is shown to the teacher, which reasons backward from it (rationalization); the training target always uses the original reference query, never the teacher’s re-generation. A separate *forward* variant for the verified-trace experiment (§5.2) hides the reference, samples $k=4$ candidate rationale–query pairs, and keeps only traces whose query *executes to the reference result* (the STaR-SQL ingredient He et al. 2025); the keep rate is 57.6%, comparable to STaR-SQL’s $\sim 50\%$ on SQL.

Training. We fine-tune `google/gemma-2-9b-it` (and `Llama-3.1-8B-Instruct` as the second family) with the QLoRA configuration of the Neo4j baseline [Ozsoy et al., 2025, Dettmers et al., 2023, Hu et al., 2021]: 4-bit NF4 with double quantization and BF16 compute, LoRA $r=64$, $\alpha=64$, dropout 0.05 on all linear projections, 1 epoch, learning rate 2×10^{-5} , effective batch 32, max sequence length 1,600. Loss is computed over response tokens only (completion-only masking); prompt tokens are masked to the cross-entropy ignore index. The direct arm’s target is the query y ; the CoT arm’s target is $r \oplus y$. This masking choice is itself a measured variable in §5.4.

Inference and metrics. Greedy decoding under the training-matched prompt for each arm (the CoT arm’s prompt requests step-by-step reasoning; the query is parsed after a “Cypher output:” marker). *Translation-based:* GLEU [Wu et al., 2016] via HuggingFace Evaluate and whitespace-normalized string exact match (EM), on the full Neo4j test set ($n=4,833$). *Execution-based:* predicted and reference queries are executed—against the public Neo4j demo databases for the Neo4j benchmark ($n=2,471$ DB-eligible instances) or a local Pole-graph container for ZOGRASCOPE—and lexicographically sorted result sets are compared for exact match. For SQL we additionally report a *canonicalized* EM (sqlglot AST normalization), which credits case, whitespace, quoting, and structural normalization. *Uncertainty:* for the central deltas we report paired bootstrap 95% confidence intervals ($B=10,000$ resamples over instances; for corpus GLEU we resample per-instance sufficient statistics, for the exact-match metrics per-instance outcomes). All reported intervals exclude zero.

Leakage discipline. After discovering that a merged-split ZOGRASCOPE setup contained 58.6% instance overlap (invalidating an earlier “length-generalization SOTA” claim, §5.2), every train/test pairing in this paper is verified for overlap at the question level *and* the (question, query)-pair level, not merely by instance ID; ID-disjointness alone misses the Neo4j benchmark’s 31.7% verbatim question leakage.

5 Results

5.1 The matched negative result on Text-to-Cypher

Table 1 reports the matched comparison on the Neo4j 2024 benchmark. For *both* model families, CoT *lowers* every translation and execution metric. A secondary apparent positive from the unmatched comparison also fails: CoT does not improve syntactic validity. The direct-answer Gemma produces *fewer* invalid queries than its CoT counterpart (100 vs. 114 execution errors); the apparent “CoT halves prediction errors” effect exists only against the published adapter’s 242.

The apparent positive that standard practice produces decomposes cleanly (Table 2): measuring the CoT model against the *published* adapter run through our inference harness—rather than against a matched control—yields “+0.1227 GLEU from CoT.” Replacing that baseline with the matched direct-answer model shows the gain was the training pipeline (+0.1399); CoT itself *subtracts* (−0.0172).

5.2 Robustness: leakage, verified traces, and generalization splits

Not a leakage artifact. Neo4j 2024 contains 31.7% of test questions verbatim in training, but the leakage is adversarial (often a *different* gold query); on genuinely unseen questions the direct-answer model beats CoT by +0.068 EM, so CoT loses *most* where leakage is absent. **Verified traces don’t rescue it.** A natural objection is that our traces are post-hoc rationalizations rather than

Table 1: Matched direct-vs-CoT on Neo4j 2024 (our pipeline; only the training target differs). CoT hurts both families. Brackets: paired bootstrap 95% CIs on the effect; all exclude zero.

Model	Target	GLEU	String EM	Exec EM
Gemma-2-9B	direct	0.7854	0.4331	0.2975
Gemma-2-9B	CoT	0.7682	0.3799	0.2554
Llama-3.1-8B	direct	0.7680	0.4223	0.2865
Llama-3.1-8B	CoT	0.7416	0.3000	0.2161
CoT effect (Gemma)		−0.0172 [−0.025, −0.010]	−0.0532 [−0.062, −0.044]	−0.0421 [−0.059, −0.025]
CoT effect (Llama)		−0.0264 [−0.039, −0.011]	−0.1223 [−0.133, −0.111]	−0.0704 [−0.087, −0.053]

Table 2: Decomposition of the apparent “+0.1227 GLEU from CoT” produced by the unmatched published-baseline comparison (Neo4j 2024).

Component	Δ GLEU
Pipeline (Neo4j published adapter $\rightarrow$ our direct-answer adapter, same inference)	+0.1399
CoT (our direct-answer $\rightarrow$ our CoT)	−0.0172
Net (apparent “CoT effect”)	+0.1227

verified reasoning. Retraining both arms on the same 3,938 execution-verified forward-generated instances (the STaR-SQL ingredient; keep rate 57.6%, comparable to STaR-SQL’s on SQL—so forward Cypher generation is not intrinsically harder) still yields CoT below direct (exec EM 0.0939 vs. 0.1052, with 60% more invalid queries); absolute numbers are low because the training set is small, but the comparison is internally matched. **Generalization.** On clean, per-split ZOGRAS-COPE (leakage-verified), direct-answer wins every split including length (Table 3)—reversing the apparent “length-generalization SOTA” that the leaked merged-split evaluation manufactures.

5.3 Cross-formalism

Table 4 extends the matched comparison to SPARQL and SQL. CoT hurts SPARQL (LC-QuAD 2.0 over Wikidata, $n=4,592$; GLEU −0.147, 95% CI [−0.158, −0.137]) as it does Cypher—and by a wider margin. Absolute SPARQL accuracy is low for both arms because Wikidata identifiers (`wd:Q...`, `wdt:P...`) are opaque, but both arms face the same entity-linking difficulty, so the delta is internally valid.

For SQL we guard against our own hypothesis (P1 inverted): string metrics undercount SQL correctness, since SQL admits many equivalent surface forms, so a string-level CoT deficit could be a metric artifact. Two tests rule that out. First, re-scoring the synthetic-SQL predictions at three levels of increasing semantic awareness—raw string EM (−0.0473), normalized EM (−0.0485), and sqlglot canonical-AST EM (−0.0477; 95% CI [−0.056, −0.039], $n=5,851$)—the deficit does not close as the metric becomes more semantic, and CoT is worse or tied in every complexity stratum, including the compositional ones. Second, and decisively, the matched protocol on *Spider with execution accuracy*—the metric the CoT-helps-SQL literature reports—returns the same sign: direct 0.7669 vs. CoT 0.6683 on the 1,034-instance dev set, a CoT effect of −0.0986 (95% CI [−0.124, −0.074]), with nearly twice the invalid queries (93 vs. 48). *In this pipeline, CoT distillation fails to help even*

Table 3: Clean ZOGRASCOPE execution accuracy (per-split, leakage-verified). Direct wins every split; paired bootstrap 95% CIs on the difference all exclude zero.

Split	n	Direct	CoT	Δ (direct – CoT) [95% CI]
IID	768	0.8255	0.6094	+0.216 [+0.184, +0.249]
Compositional	1349	0.6612	0.4181	+0.243 [+0.213, +0.274]
Length (dist. shift)	1253	0.4397	0.2306	+0.209 [+0.178, +0.241]

Table 4: Matched CoT effect across formalisms (our pipeline), with paired bootstrap 95% CIs. Negative everywhere, including SQL under the literature’s own execution-accuracy metric.

Formalism (dataset)	Metric	CoT effect	95% CI
Cypher (Neo4j 2024)	GLEU	−0.0172	[−0.025, −0.010]
SPARQL (LC-QuAD 2.0)	GLEU	−0.1470	[−0.158, −0.137]
SQL (synthetic)	canon. EM	−0.0477	[−0.056, −0.039]
SQL (Spider)	execution acc.	−0.0986	[−0.124, −0.074]

the compositional formalism it was designed for. Notably, our direct-answer Gemma-2-9B reaches 76.7% Spider dev execution accuracy—above the 75.0% that STaR-SQL reports for CoT-fine-tuned Llama-3.1-8B and well above its 68.6% direct-SFT baseline [He et al., 2025]—a soft comparison across base models, but one that locates our pipeline’s direct arm at the strong end of the published range.

5.4 What does transfer: execution-grounded selection and a stronger recipe

Selection (P1, P2). Consistent with P1, string-vote self-consistency at five samples falls *below* single-sample greedy decoding (0.2509 vs. 0.2554 exec EM): Cypher’s near-canonical output space leaves nothing to vote across, so sampling only adds noise. Consistent with P2, clustering the same five candidates by their *execution results* (MBR selection) rises above both (0.2665; Table 5): semantically-equivalent-but-textually-different candidates collapse into one result cluster and reinforce each other. The oracle ceiling (any of the five candidates correct) is 0.4302. The evaluated instances partition three ways: 57% are generation-bound (no correct candidate among the five—only more sampling or a stronger base model can help), 27% are solved by MBR’s result-cluster vote, and 16% are verifier-addressable (a correct candidate exists but loses the vote; in 77% of these it is the lone correct candidate). A trained verifier in the STaR-SQL/ORM style is thus the natural future extension, with its expected gain bounded by that 16%.

Training recipe. The second surviving positive is mundane but large: our direct-answer model exceeds the published fine-tuned baseline by +0.23 GLEU (0.7854 vs. 0.5560). This decomposes into +0.09 of *evaluation harness* (the same published weights score 0.6455 under our inference vs. 0.5560 published; GLEU is also highly sensitive to tokenization for punctuation-dense Cypher) and +0.14 of *training*. The configurations differ in one decisive place: the published recipe trains with full-sequence loss over schema-heavy prompts (with a $\sim 30:1$ schema-to-answer token ratio, $\sim 97\%$ of the gradient reconstructs schemas), while ours masks the prompt and spends the entire loss on the query. A controlled full-sequence ablation of our own pipeline attributes +0.044 GLEU to the masking choice alone (0.7854 completion-only vs. 0.7415 full-sequence), and adding sequence packing—thereby reproducing the published recipe’s documented training choices in full—costs only a further

Table 5: Self-consistency selection on the CoT model (Neo4j exec subset). Execution-result voting transfers; string voting does not.

Selection method	Exec EM
String-vote SC@5	0.2509
Greedy (single)	0.2554
Execution-vote SC@5 (MBR)	0.2665
Oracle (any of 5 correct)	0.4302

0.009 (0.7330). The documented recipe knobs thus account for ≈ 0.05 of the +0.14 training gap; the remaining ≈ 0.09 persists even when the published recipe is retrained in our own pipeline and evaluated identically (our recipe-reproduction model still scores 0.7330 vs. the published artifact’s 0.6455 under the same inference), placing the residual at the implementation level—framework and library versions, data formatting—rather than in any documented hyperparameter, and we flag it as such. The harness component resists documentation-level reproduction symmetrically: the natural hypothesis for the published 0.5560—inference truncated at the training length of 1,600 tokens—collapses corpus GLEU to 0.196 when tried (truncation deletes the output cue from the $\sim 9\%$ of prompts that exceed the limit; those generations run to the token cap and dominate the corpus-level denominator, while the untruncated remainder scores 0.67), so no plain truncation setting yields the published number either. This sharpens the paper’s methodological point from the other side: neither the published artifact (from its documented recipe) nor its published score (from its documented inference) was reproducible—even before the artifact was mistaken for a control. The practical point stands independently: completion-only masking on long-schema inputs is a cheap, reusable recipe improvement, and the model it produces is—to our knowledge—the strongest open-weight result on this benchmark under our harness, with the important caveat that cross-harness absolute comparisons are unreliable (which is itself one of this paper’s lessons).

6 Mechanism: why CoT hurts Cypher

6.1 The failure is compositional, not relational

On clean ZOGRASCOPE with execution ground truth, we analyze the 956 instances where the direct model is execution-correct and the CoT model is not. Only 3% of these failures use relationships unrelated to the reference—*97% identify the correct relationships*. Schema linking (the InterCOL step) works. The failure is in assembly: 49% use the right relationships in the wrong connected structure (wrong order, misplaced filters, or fragmentation of one connected traversal into disconnected MATCH clauses), 32% drop hops, 5% add hops. The model has the right pieces and puts them together wrong—precisely what P3 predicts for a globally-coupled output.

6.2 Causal test: decomposition drives fragmentation

If decomposition is the assembly-breaker, a reasoning format that keeps the reasoning but removes the decomposition should recover part of the penalty. We trained a *holistic-path* CoT variant whose trace states the full connected path explicitly, with no sub-question decomposition, and an *enumeration* variant (E4) that additionally lists and counts every required relationship before constructing. Table 6 reports the ladder. Holistic-CoT beats QDecomp-CoT on every split (+4.6 to +10.0pp), recovering $\sim 40\%$ of the CoT penalty, and the gain is localized: fragmentation of connected paths

Table 6: Reasoning-format ablation (ZOGRASCOPE, execution accuracy, common instances). Holistic framing recovers part of the CoT penalty; no format closes the gap to direct.

Config	Length	IID	Comp.
Direct-answer	0.475	0.826	0.661
QDecomp-CoT	0.258	0.609	0.418
Holistic-CoT	0.303	0.710	0.511
Enum-CoT (E4)	0.231	0.669	0.429

drops from 11% to 2% (length split) and 6% to 1% (regular)—a 5–6 $\times$ reduction. Decomposition is thus *causally* a major component of the harm. Two boundaries: no reasoning format closes the gap to direct-answer (holistic still trails by 12–17pp), and *more* reasoning structure backfires—E4’s enumeration scaffolding underperforms holistic everywhere and partially reintroduces fragmentation. Minimal reasoning is the best reasoning; none is better still.

6.3 The residual: error accumulation on a copy-and-place task

What explains the rest of the penalty? Not path truncation: although the reasoning does commit to too-short paths on long traversals (and does so at the planning stage—in 99–100% of dropped-hop failures the hop is already missing from the trace), truncation *alone* accounts for only 2% of holistic-CoT failures, and E4 behaviorally confirmed its insufficiency by fixing short-path hop deficits without improving accuracy. The robust CoT-specific signature is *filter-value corruption*: the CoT model filters on wrong values 3.7–5 $\times$ more often than the direct model (16% vs. 4% of length-split predictions), and in 92% of these cases the wrong value already appears in the reasoning trace—corrupted during reasoning, then faithfully copied into the query (e.g., “vehicle-related crimes in BL5” acquiring a spurious “**vehicle**” filter). ZOGRASCOPE hands entity values to the model, making generation a copy-and-place task (§3): the direct model copies each value once; the CoT model re-states values, paths, and structure in prose first, and each re-statement risks corruption. This error-accumulation account unifies the ladder’s monotonicity—more reasoning is strictly worse—though we note its bounds honestly: filter corruption touches only $\sim$ 16% of length-split predictions, and the remaining deficit is diffuse (right relationships, right values, subtly wrong assembly).

7 Discussion

One property, three predictions. The results cohere around a single structural property of the output space. Because a Cypher pattern has essentially one canonical surface form, sampling produces no useful diversity (P1: string voting falls below greedy) and there is little for a reasoning process to explore or for a selector to choose among; because that form is a globally coupled connected object, decomposing it fragments it (P3: causally shown in §6); and because execution results abstract away from surface form entirely, execution-grounded selection is the transfer that survives (P2). The same property explains all three outcomes—which is what makes it a hypothesis rather than a post-hoc grouping.

Reconciling with the SQL literature: the baseline-strength account. The Spider control sharpens the story. One might have expected the structural hypothesis to imply a sign flip—CoT helping compositional SQL while hurting holistic Cypher—but in our pipeline CoT hurts

SQL too, under the literature’s own execution-accuracy metric (-0.099 on Spider dev). Three observations reconcile this with the published CoT-helps-SQL results. First, the clean evidence there is thinner than its headlines: STaR-SQL’s same-data CoT-vs-direct gap is $+6.4\text{pp}$ (the celebrated $+18\text{pp}$ includes best-of-16 verification), and other celebrated results bundle CoT with RL or DPO, or prompt frontier models rather than fine-tune small ones. Second, our direct baselines are unusually strong: the completion-only recipe puts our direct-SFT Gemma at 76.7% Spider dev execution accuracy—above STaR-SQL’s *CoT*-fine-tuned 75.0% . Under this *baseline-strength* account, published CoT-SFT gains partly measure the weakness of their direct-answer controls, and the headroom reasoning could occupy closes as the direct recipe improves. Third, the structural hypothesis retains exactly its directly-tested content: the selection asymmetry (P1/P2, a Cypher-specific consequence of the canonical output space) and the causal fragmentation mechanism (P3, demonstrated by ablation on graph patterns). What the Spider result removes is any temptation to read P3 as “decomposition therefore helps SQL”—the sign of the CoT effect everywhere we measure is governed by baseline strength, while the structure of the output space governs the *failure modes* (fragmentation for graphs) and *which remedies work* (execution grounding, not string diversity). We deliberately refrain from comparing penalty magnitudes across formalisms: the benchmarks differ in task character, scale, and metric, so only the within-benchmark sign and the mechanism are portable conclusions.

A published artifact is not a control. The unmatched comparison that standard practice licenses reports $+0.1227$ GLEU from CoT (Table 2). Its baseline—a *published* fine-tuned adapter run through our inference harness—differs from the CoT model in training pipeline and inference context, not just in training target, and the matched direct-answer control reverses the sign. A leaked cross-split evaluation likewise manufactures a “length-generalization SOTA” (§5.2) that instance-ID checks pass. Neither error is exotic: one borrows a control, the other trusts ID-disjointness, and both are common practice. The lesson is uncomfortable and general: *deltas attributed to an intervention are only meaningful against a baseline trained in the same pipeline*, and split hygiene must be verified at the content level, not the ID level. We demonstrate both failure modes in full detail precisely because they are cheap to fall into and expensive to detect.

8 Limitations

SQL scope. The SQL negative rests on one synthetic dataset (string and canonical-AST metrics) and one Spider run (execution accuracy); Spider dev is the standard but not the hardest SQL benchmark, and harder value-inference-heavy benchmarks (e.g. BIRD) could in principle give reasoning more to contribute. Our claim is therefore that CoT-SFT does not help SQL *in this matched strong-baseline pipeline*, not that it never helps SQL. **Single teacher and scale.** All traces come from one teacher (GPT-oss-120B) in one format family, and both student families are 8–9B models; a different teacher, trace style, or scale could shift the deltas, though the holistic/enumeration ladder shows the effect is not an artifact of one trace format. **Benchmark leakage.** Neo4j 2024 contains 31.7% verbatim question leakage (adversarial in direction—leaked questions often carry different gold queries—which we exploit as evidence in §5.2); absolute numbers on this benchmark should be read with that in mind. **Live-database drift.** Neo4j execution evaluation runs against public demo databases whose state changes over time (reference-failure counts varied from 166 to 51 between our evaluation dates); within-run comparisons are unaffected, but cross-run execution numbers carry a small same-denominator caveat. **Uncertainty.** Every reported CoT delta carries a paired bootstrap 95% CI, and all twelve exclude zero (Tables 1, 3, and 4). Each arm is,

however, a single training run, so seed-to-seed training variance is not measured; the effect’s directional consistency across twelve comparisons, two model families, and three formalisms is the practical guard against a single-seed artifact. **Task character.** ZOGRASCOPE supplies entity values in the question, making it a copy-and-place task; benchmarks requiring value inference may dilute (or reverse) the error-accumulation component of the mechanism, though the fragmentation component is task-independent.

9 Conclusion

Which Text-to-SQL techniques transfer to Text-to-Cypher? Under matched-pipeline controls, two transfer: execution-grounded selection (result-clustering MBR beats both greedy decoding and string voting) and a careful SFT recipe (completion-only loss masking on schema-heavy prompts yields a direct-answer model far above the published baseline). Chain-of-thought distillation—the highest-profile candidate—does not: it modestly degrades Text-to-Cypher across two model families, across naive and execution-verified traces, and across in-distribution, compositional, and length-generalization regimes; string-diversity self-consistency fails alongside it, and matched controls return the same sign for SPARQL and for SQL itself on Spider under execution accuracy. The constrained-output-space hypothesis explains the Cypher-specific pattern mechanistically: connected graph patterns are globally coupled objects with near-canonical surface forms, so decomposition fragments them (causally shown), reasoning-first generation accumulates copy errors, and sampling yields no diversity worth voting over—while execution results, which bypass surface form entirely, remain a useful signal. The uniform sign across formalisms points to a complementary cause: against a strong direct-answer recipe, reasoning distillation has no headroom left to occupy. The broader lesson is methodological: the large apparent CoT gain that standard practice produces is an artifact of borrowing a published model as a control, and it takes a matched in-pipeline baseline to find the sign of the true effect. Toolboxes do not transfer by default; controls do not transfer at all.

References

- Francesco Cazzaro, Justin Kleindienst, Sofia Márquez Gomez, and Ariadna Quattoni. ZOGRASCOPE: A new benchmark for semantic parsing over property graphs. *arXiv preprint arXiv:2503.05268*, 2025.
- Vashu Chauhan et al. Mind the query: Large language models for cypher query generation. In *EMNLP 2025 Industry Track*, 2025.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *NeurIPS 2023*, 2023. arXiv:2305.14314.
- G Feng, G Zhu, S Shi, Y Sun, Z Fan, S Gao, and J Hu. Robust nl-to-cypher translation for kbqa: Harnessing large language model with chain of prompts. In *CCKS 2023, Springer LNCS*, 2023.
- Yanlin Feng, Simone Papicchio, and Sajjadur Rahman. CypherBench: Towards precise retrieval over full-scale modern knowledge graphs in the LLM era. *ACL 2025*, 2025. arXiv:2412.18702.
- Mingqian He, Yongliang Shen, Wenqi Zhang, Qiuying Peng, Jun Wang, and Weiming Lu. Star-sql: Self-taught reasoner for text-to-sql. In *ACL 2025*, 2025. arXiv:2502.13550.

- Markus Hornsteiner et al. Real-time text-to-cypher query generation with large language models for graph databases. *Future Internet*, 16(12):438, 2024.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- M Kobeissi, N Assy, W Gaaloul, B Defude, and B Haidar. An intent-based natural language interface for querying process execution data. In *ICPM 2021*, 2021.
- Xinyu Liu et al. A survey of nl2sql with large language models. *arXiv preprint arXiv:2408.05109*, 2024.
- Zhaoyang Liu et al. Uncovering the impact of chain-of-thought reasoning for direct preference optimization. *ACL 2025*, 2025. arXiv:2502.11656.
- Aidin Mohammadjafari et al. From natural language to sql: Review of llm-based text-to-sql systems. *arXiv preprint arXiv:2410.01066*, 2024.
- Lunyu Nie et al. Graphq ir: Unifying the semantic parsing of graph query languages with one intermediate representation. In *EMNLP 2022*, 2022.
- Makbule Gulcin Ozsoy. Exploring iterative refinement for text2cypher. *Medium (Neo4j Developer Blog)*, 2025a.
- Makbule Gulcin Ozsoy. Text2cypher: Data pruning using hard example selection. *arXiv preprint arXiv:2505.05122*, 2025b.
- Makbule Gulcin Ozsoy. Enhancing text2cypher with schema filtering. *arXiv preprint arXiv:2505.05118*, 2025c.
- Makbule Gulcin Ozsoy. Adapter fusion for multilingual text2cypher. *arXiv preprint arXiv:2601.16097*, 2026.
- Makbule Gulcin Ozsoy, Leila Messallem, Jon Besga, and Gianandrea Minneci. Text2cypher: Bridging natural language and graph databases. *Proceedings of the GenAIK Workshop (COLING 2025)*, pages 100–108, 2025. arXiv:2412.10064.
- Mohammadreza Pourreza and Davood Rafiei. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. In *NeurIPS 2023*, 2023. arXiv:2304.11015.
- Mohammadreza Pourreza and Davood Rafiei. Dts-sql: Decomposed text-to-sql with small large language models. *EMNLP 2024 Findings*, 2024. arXiv:2402.01117.
- Mohammadreza Pourreza et al. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *ICLR 2025*, 2025a. arXiv:2410.01943.
- Mohammadreza Pourreza et al. Reasoning-sql: Reinforcement learning with partial rewards for text-to-sql. *arXiv preprint arXiv:2503.23157*, 2025b.
- Gaetano Rossiello et al. Rationalization models for text-to-sql. *ICLR 2025 Workshop on Reasoning and Planning for LLMs*, 2025. arXiv:2502.06759.

- Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Josefa Lia Stoisser, Marc Bounovski Martell, Lawrence Phillips, Casper Hansen, and Julien Fauqueur. STRuCT-LLM: Unifying tabular and graph reasoning with reinforcement learning for semantic parsing. *arXiv preprint arXiv:2506.21575*, 2025.
- Chang-You Tai, Ziru Chen, Tianshu Zhang, Xiang Deng, and Huan Sun. Exploring chain of thought style prompting for text-to-sql. In *EMNLP 2023*, 2023. arXiv:2305.14215.
- Parth Thaker and Guy Bresler. Struct-sql: Structured cot distillation. *arXiv preprint arXiv:2512.17053*, 2025.
- Aman Tiwari, Shiva Krishna Reddy Malay, Vikas Yadav, Masoud Hashemi, and Sathwik Tejaswi Madhusudhan. Auto-cypher: Improving llms on cypher generation via llm-supervised generation-verification framework. In *NAACL 2025*, 2025. arXiv:2412.12612.
- Quoc-Bao-Huy Tran, Aagha Abdul Waheed, Syed Mudassir, and Sun-Tae Chung. Refining text2cypher on small language model with reinforcement learning leveraging semantic information. *Applied Sciences*, 15(15):8206, 2025.
- Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. MAC-SQL: A multi-agent collaborative framework for text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025. arXiv:2312.11242.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS 2022*, 2022.
- Tomer Wolfson, Mor Geva, Ankit Gupta, Matt Gardner, Yoav Goldberg, Daniel Deutch, and Jonathan Berant. Break it down: A question understanding benchmark. *TACL*, 2020. arXiv:2001.11770.
- Tomer Wolfson et al. Weakly supervised text-to-sql parsing through question decomposition. *NAACL 2022*, 2022. arXiv:2112.06311.
- Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- Chao Yang, Changyi Li, Xiaodu Hu, Hao Yu, and Jinzhi Lu. Enhancing knowledge graph interactions: A comprehensive text-to-cypher pipeline with large language models. *Information Processing and Management*, 63(1):104280, 2026.
- Zhewei Yao et al. Arctic-text2sql-r1. *arXiv preprint arXiv:2505.20315*, 2025a.
- Zhewei Yao et al. Excot: Iterative cot with dpo for text-to-sql. *ACL 2025 Findings*, 2025b. arXiv:2503.19988.